Classical Conditioning and Reinforcement Learning

Euntaek Lee

Seoul National University

May 12, 2023

Table of contents

1. Classical conditioning
2. Static Action Choice
3. Sequential Action Choice
4. Generalized sequential Action choice

Pavlovian experiment

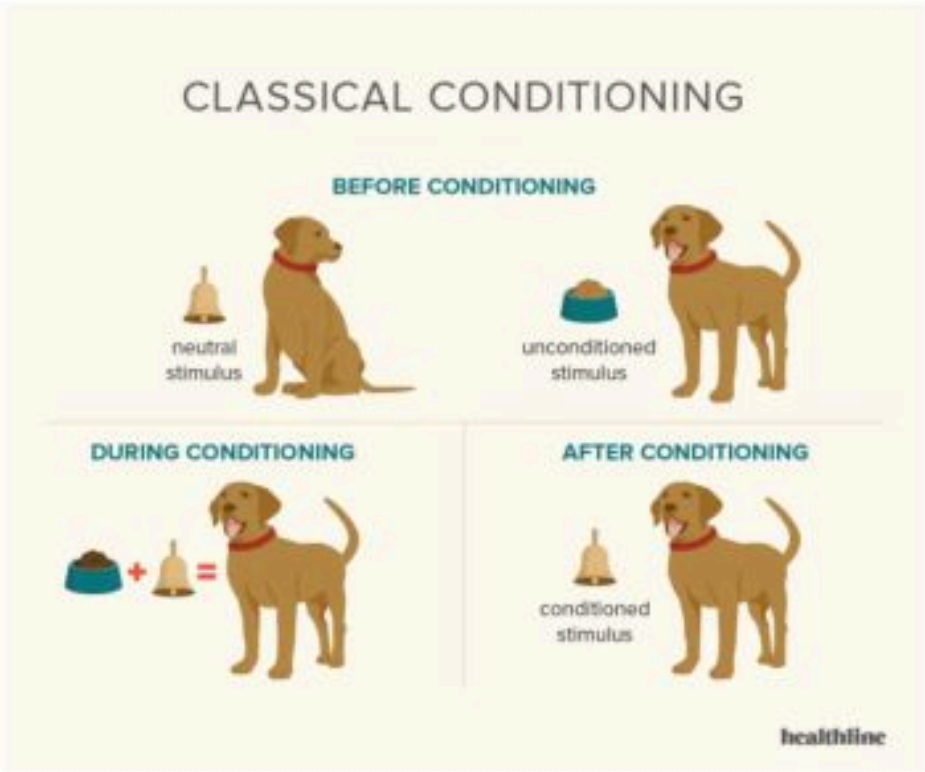

Figure: Pavlovian experiment[1]

[1] https://www.healthline.com/health/classical-conditioning

Rescorla-Wagner rule

Assumption

$$(\text{expected reward } v) = (\text{weight } w) \times (\text{stimulus } u)$$

Rescorla-Wagner rule

$$w \to w + \varepsilon \delta u, \quad \delta = r - v$$

for given reward r.

Numeric result

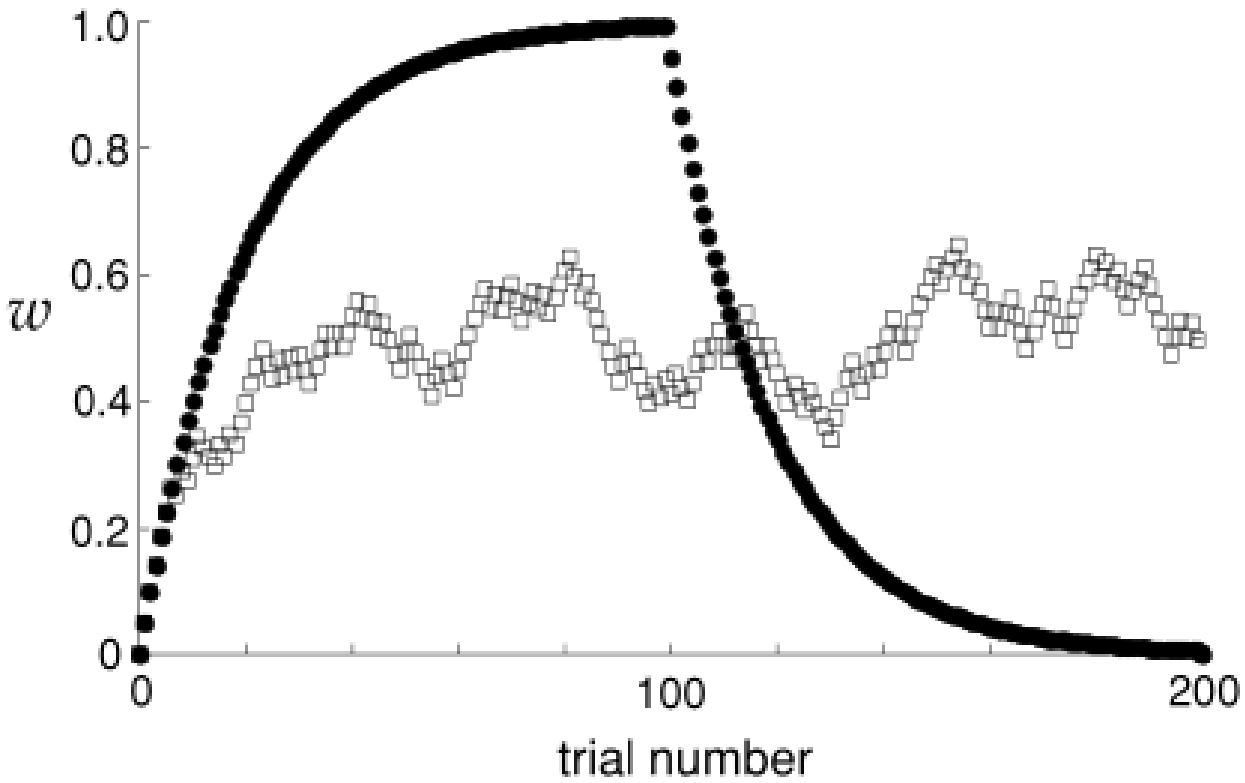

Figure: Rescorla-Wagner model

Rescorla-Wagner rule

For multiple types of stimuli, we consider multidimensional generalization of R-W rule by

$$v = \boldsymbol{w} \cdot \boldsymbol{u}, \quad \delta = r - v, \quad \boldsymbol{w} \to \boldsymbol{w} + \varepsilon\delta\boldsymbol{u}$$

for given reward r. This can be used for describing some classical conditioning paradigms.

Conditioning paradigms

Paradigm	Pre-train	Train	Result
Pavlovian		$s \to r$	$s \to (r)$
Extinction	$s \to r$	$s \to \cdot$	$s \to 0$
Partial		$s \to r$, $s \to \cdot$	$s \to \alpha(r)$
Blocking	$s_1 \to r$	$s_1 + s_2 \to r$	$s_1 \to (r)$, $s_2 \to 0$
Inhibitory		$s_1 + s_2 \to \cdot$, $s_1 \to r$	$s_1 \to (r)$, $s_2 \to -(r)$
Overshadow		$s_1 + s_2 \to r$	$s_1 \to \alpha_1(r)$, $s_2 \to \alpha_2(r)$
Secondary	$s_1 \to r$	$s_2 \to s_1$	$s_2 \to (r)$

Table: Classical conditioning paradigms

Blocking

- Pre-trained : $s_1 \rightarrow r$,
- Train : $s_1 + s_2 \rightarrow r$,
- Result : $s_1 \rightarrow (r)$, $s_2 \rightarrow 0$

$$v = \boldsymbol{w} \cdot \boldsymbol{u}, \quad \delta = r - v, \quad \boldsymbol{w} \rightarrow \boldsymbol{w} + \varepsilon \delta \boldsymbol{u}$$

Secondary

- Pre-train : $s_1 \rightarrow r$
- Train : $s_2 \rightarrow s_1$
- Result : $s_2 \rightarrow (r)$

Temporal Difference Learning

Here we consider

$$u = u(t), v = v(t), r = r(t), \quad 0 \leq t \leq T.$$

We suggest

$$\left\langle \sum_{\tau=0}^{T-t} r\left(t+\tau\right) \right\rangle = v(t) \approx \sum_{\tau=0}^{t} w\left(\tau\right) u\left(t-\tau\right)$$

Temporal Difference Learning

and update $w(t)$ by

$$w(\tau) \to w(\tau) + \varepsilon\delta(t)u(t-\tau)$$

with

$$\delta(t) := \sum_{\tau=0}^{T-t} r(t+\tau) - v(t).$$
$$= (\text{Full potential reward}) - (\text{predicted reward})$$

Temporal Difference Learning

and update $w(t)$ by

$$w(\tau) \to w(\tau) + \varepsilon\delta(t)u(t-\tau)$$

with

$$\delta(t) := \sum_{\tau=0}^{T-t} r(t+\tau) - v(t).$$

By (9.8)-(9.10) in textbook, we can simplify $\delta(t)$ by

$$\delta(t) = r(t) + v(t+1) - v(t).$$

Numeric simulation

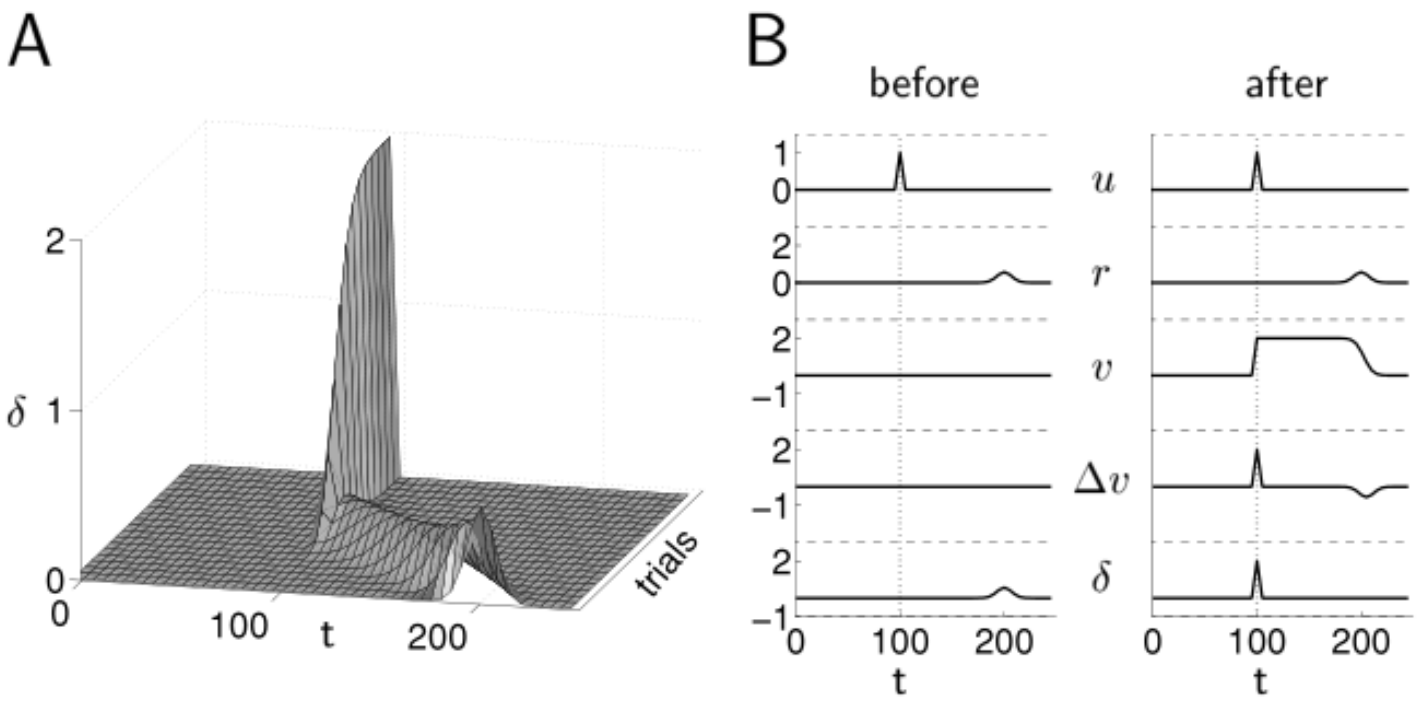

Figure: Learning to predict a reward

Temporal Difference Learning

Remark

We can explain the secondary conditioning with TDL.

Experiment with bee

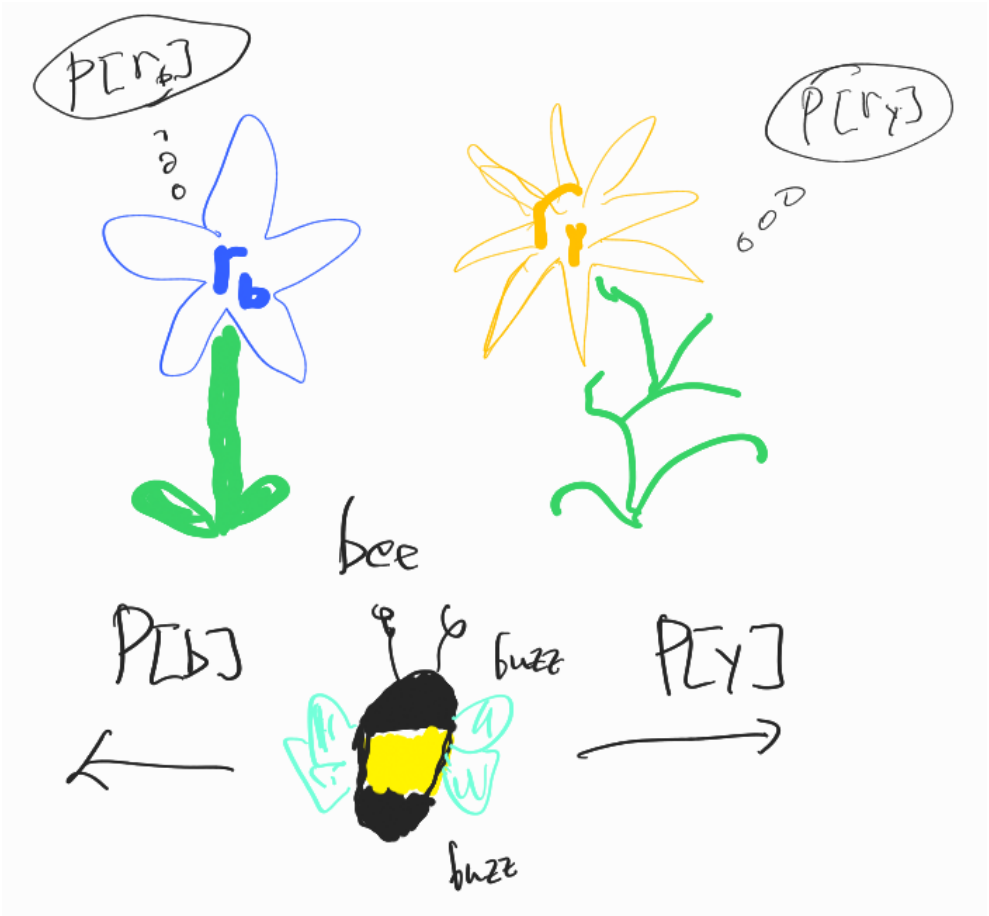

Figure: Experiment with bee

Policy of bee

Assume the bee chooses blue and yellow flowers with probability

$$P[\mathrm{b}] = \frac{\exp(\beta m_{\mathrm{b}})}{\exp(\beta m_{\mathrm{b}}) + \exp(\beta m_{\mathrm{y}})}, \quad P[\mathrm{y}] = \frac{\exp(\beta m_{\mathrm{y}})}{\exp(\beta m_{\mathrm{b}}) + \exp(\beta m_{\mathrm{y}})}.$$

We can represent it by

$$P[\mathrm{b}] = \sigma(\beta(m_{\mathrm{y}} - m_{\mathrm{b}})), \quad P[\mathrm{y}] = \sigma(\beta(m_{\mathrm{b}} - m_{\mathrm{y}}))$$

for standard sigmoid function

$$\sigma(m) = \frac{1}{1 + \exp(-m)}.$$

Question

How does a bee learn optimal parameter?

The indirect actor

Here, "Indirect" means learn

$$m_{\mathrm{b}} = \langle r_{\mathrm{b}} \rangle, \quad m_{\mathrm{y}} = \langle r_y \rangle$$

without using probability by δ-method

$$\begin{cases} m_{\mathrm{b}} \to m_{\mathrm{b}} + \epsilon\delta, & \delta = r_{\mathrm{b}} - m_{\mathrm{b}} \quad \text{lands b} \\ m_{\mathrm{y}} \to m_{\mathrm{y}} + \epsilon\delta, & \delta = r_{\mathrm{y}} - m_{\mathrm{y}} \quad \text{lands y} \end{cases}$$

The direct actor

Here, "direct" means learn

$$m_{\mathrm{b}} = \langle r_{\mathrm{b}} \rangle, \quad m_{\mathrm{y}} = \langle r_y \rangle$$

with using probability with

$$\begin{cases} m_{\mathrm{b}} \to m_{\mathrm{b}} + \epsilon \left(\delta_{a\mathrm{b}} - P\left[b \right] \right) \left(r_a - \bar{r} \right) \\ m_{\mathrm{b}} \to m_{\mathrm{y}} + \epsilon \left(\delta_{a\mathrm{y}} - P\left[y \right] \right) \left(r_a - \bar{r} \right) \end{cases}$$

where a is the action selected, $\bar{r}$ is a parameter.

Derivation

We can derive

$$\frac{\partial \langle r \rangle}{\partial m_{\mathrm{b}}} = \beta P[\mathrm{b}]((1 - P[\mathrm{b}])(\langle r_{\mathrm{b}} \rangle - \bar{r})) - \beta P[\mathrm{y}](P[\mathrm{b}](\langle r_{\mathrm{y}} \rangle - \bar{r}))$$

and if we update m_{b} by

$$m_{\mathrm{b}} \to \begin{cases} m_{\mathrm{b}} + ((1 - P[\mathrm{b}])(\langle r_{\mathrm{b}} \rangle - \bar{r})) & (\mathrm{b}) \\ m_{\mathrm{b}} - (P[\mathrm{b}](\langle r_{\mathrm{y}} \rangle - \bar{r})) & (\mathrm{y}), \end{cases}$$

with this process, the expectation of $(m_{\mathrm{b}}, m_{\mathrm{y}})$ is the direction of gradient $\left(\frac{\partial \langle r \rangle}{\partial m_{\mathrm{b}}}, \frac{\partial \langle r \rangle}{\partial m_{\mathrm{y}}}\right)$.

Comparison

- The indirect method has better performance.
- The direct actor will be useful later.

Generalization

For multivariable case, the softmax policy is

$$P[a] = \frac{\exp(\beta m_a)}{\sum_{a'=1}^{N_a} \exp(\beta m_{a'})}, \quad a = \{1, 2, \ldots, N_a\},$$

the direct actor learning rule is

$$m_{a'} \rightarrow m_{a'} + \epsilon \left(\delta_{aa'} - P\left[a'\right]\right)(r_a - \bar{r}), \quad a' \in 1, 2, \ldots, N_a$$

for chosen action a.

The maze task

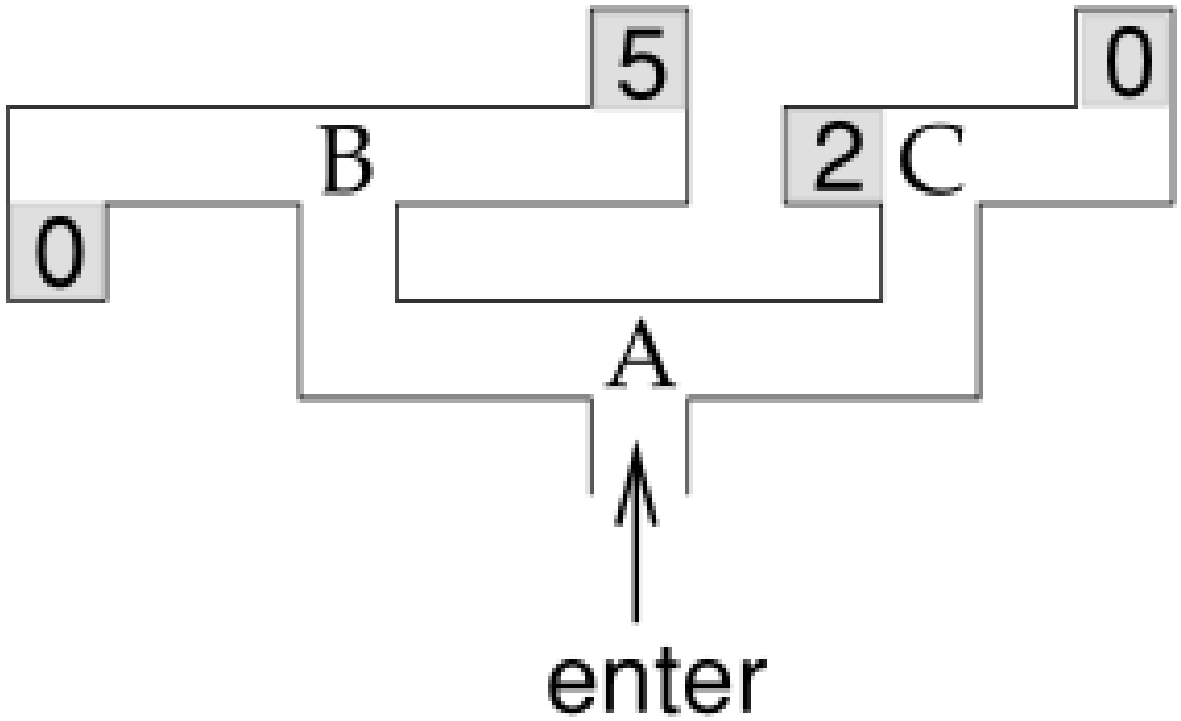

Figure: The maze task

Actor-critic learning

- $\boldsymbol{m} : \{A, B, C\} \to \mathbb{R}^2$ for $u \in \{A, B, C\}$,

  $$\boldsymbol{m}(u)_1 = \sigma\left(\beta(m_{u,\mathrm{L}} - m_{u,\mathrm{R}})\right), \quad \boldsymbol{m}(u)_2 = \sigma\left(\beta(m_{u,\mathrm{L}} - m_{u,\mathrm{R}})\right)$$

- $r_a(u) : \{L, R\} \times \{A, B, C\} \to \{0, 2, 5\}$, The immediate reward.

- $w(u) = v(u) : \{A, B, C\} \to \mathbb{R}$, the weight and predicted reward.

Learning procedure

- Step 1 : Policy evaluation, improve $w = v$
- Step 2 : Policy improvement, improve $\boldsymbol{m}$.

Policy evaluation

- To learn : $v(B) = 2.5, v(C) = 1, v(A) = 1.75$.
- Learning rule:

$$w(u) \to w(u) + \varepsilon\delta, \quad \delta = r_a(u) + v(u') - v(u)$$

if the rat chooses the action

$$a \in \{L, R\}, \quad u \xrightarrow{a} u'.$$

Numeric simulation

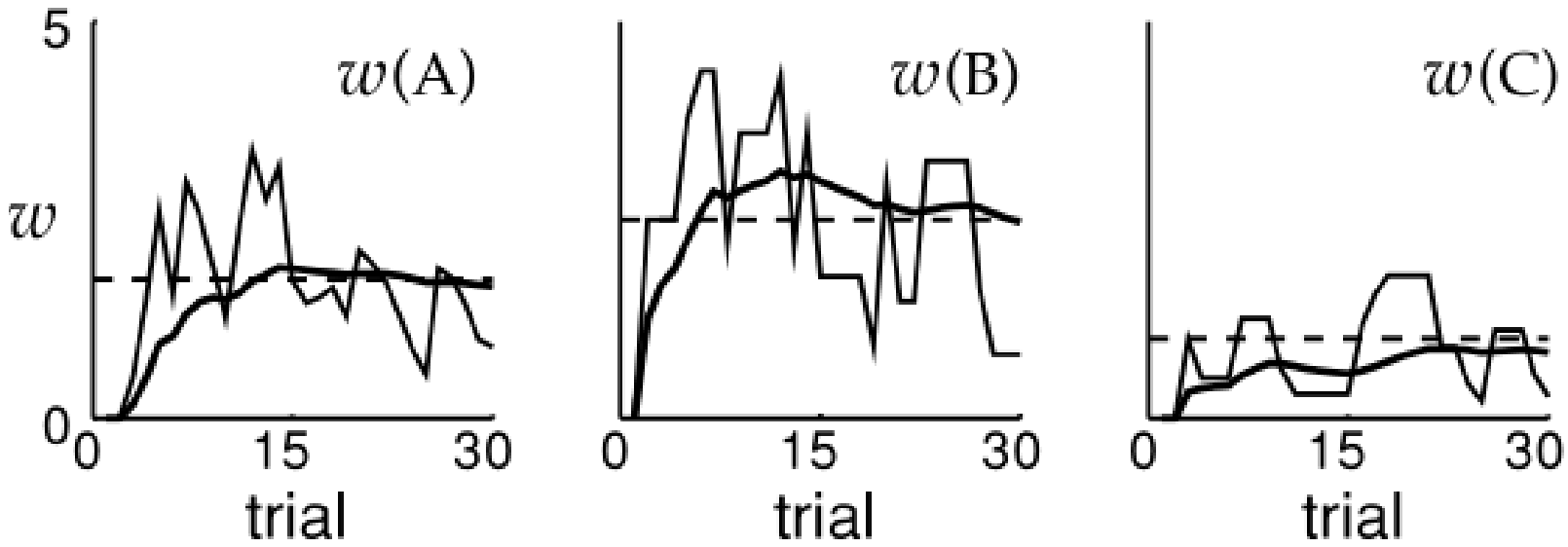

Figure: Policy evaluation

Policy improvement

Lending direct actor policy gives

$$m_{a'} \to m_{a'} + \epsilon \left(\delta_{aa'} - P\left[a'; u\right]\right)(r_a - \bar{r})$$

and we obtain

$$m_{a'} \to m_{a'} + \epsilon \left(\delta_{aa'} - P\left[a'; u\right]\right)\delta$$

where

$$\delta = r_a(u) + v(u') - v(u).$$

Numeric simulation

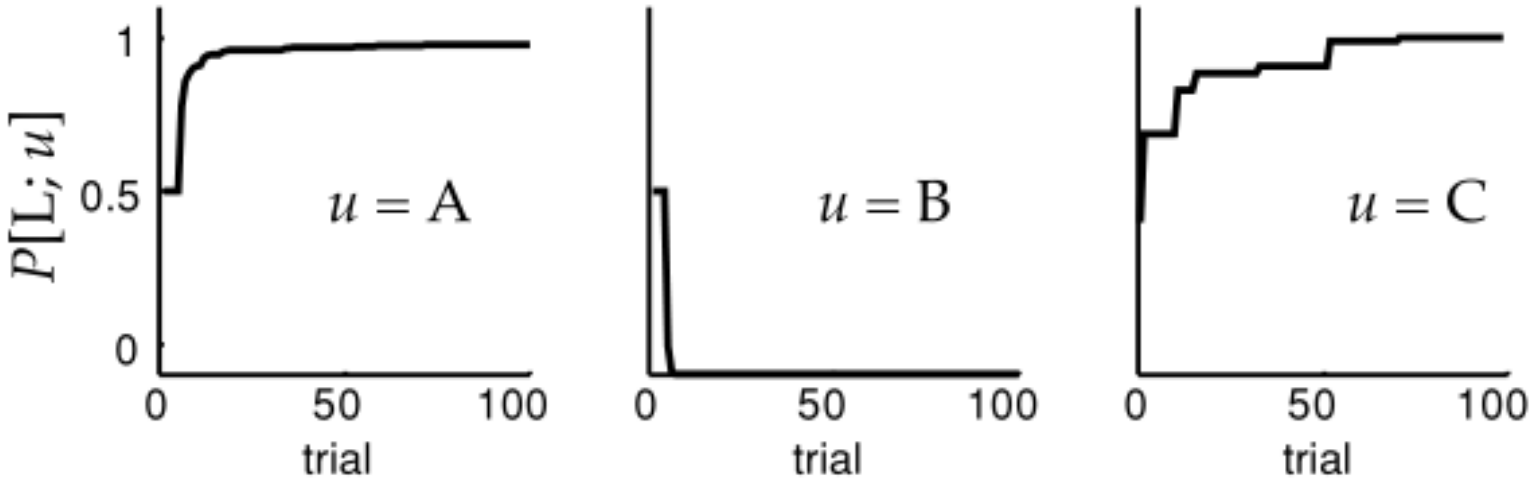

Figure: Actor-critic learning

Multidimensional A-C

- $\mathcal{U}$: the set of location.
- $\boldsymbol{m} : \mathcal{U} \to \mathbb{R}^{|\mathcal{A}|}$, output is softmax probability density.
- $r_a(u) : \mathcal{A} \times \mathcal{U} \to \mathcal{R}$, The immediate reward.
- $\boldsymbol{u} : \mathcal{U} \to \mathbb{R}^d$, the state vector.
- $\boldsymbol{w} : \mathcal{U} \to \mathbb{R}^d$, the weight.
- $v = \boldsymbol{w} \cdot \boldsymbol{u}(u)$, the predicted value.

Learning rule

- Step 1 : Policy evaluation, improve $\boldsymbol{w}$ by

$$\boldsymbol{w} \to \boldsymbol{w} + \varepsilon\delta\boldsymbol{u}(u), \quad \delta = r_a(u) + v(u') - v(u).$$

- Step 2 : Policy improvement, improve

$$M_{a'b} \to M_{a'b} + \varepsilon\left(\delta_{aa'} - P\left[a'; u\right]\right)\delta u_b(u).$$

For

$$\boldsymbol{m} = \boldsymbol{M} \cdot \boldsymbol{u}(u) \text{ for } \boldsymbol{M} \in M_{|\mathcal{A}|\times d}\left(\mathbb{R}\right).$$

Reward-Punishment

- Step 1 : Policy evaluation, improve $\boldsymbol{w}$ by

$$\boldsymbol{w} \to \boldsymbol{w} + \varepsilon\delta\boldsymbol{u}(u), \quad \delta = r_a(u) + \gamma v(u') - v(u).$$

- Step 2 : Policy improvement, improve

$$M_{a'b} \to M_{a'b} + \varepsilon\left(\delta_{aa'} - P\left[a'; u\right]\right)\delta u_b(u).$$

Water maze task

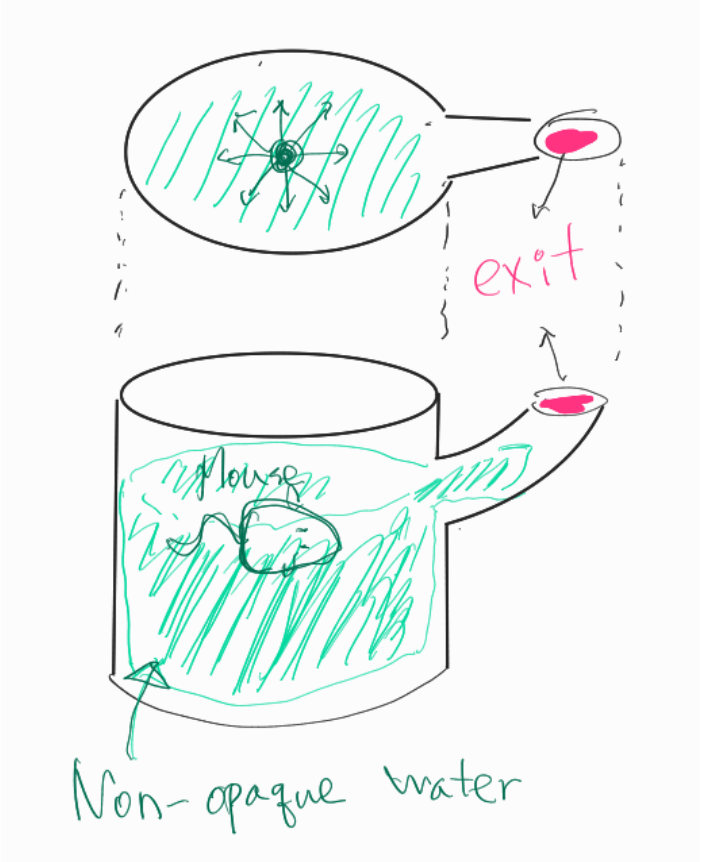

Figure: The maze task

Numeric simulation

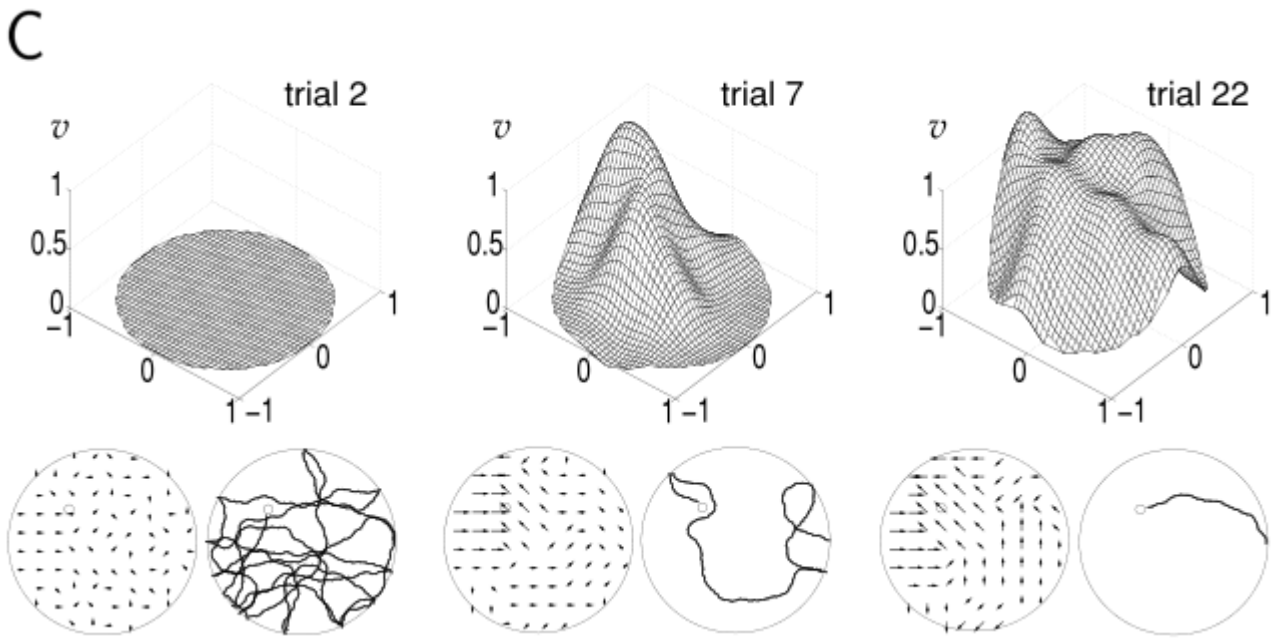

Figure: Simulation of A-C procedure

Thank you for listening!